

Dr KANGA Koffi

Table des matières

A- Les tableaux Java - (Java array)	3
1 - Déclaration des tableaux en Java	3
2 - Accéder aux éléments d'un tableau	3
3 - Changer un élément du tableau	4
4 - Longueur d'un tableau Java	4
4 - Boucle à travers un tableau	4
4-1 Lecture des éléments d'un tableau en utilisant la fonction length()	4
4-2 Lecture des éléments d'un tableau en utilisant la boucle for et une variable string	5
5 - Tableaux multidimensionnels	5
B- Classes et méthodes Java	7
1 - Notion d'objets et de classes	7
1 - 1 Exemple d'une classe prédéfinie : classe String	7
2 - Les classes et les objets Java	7
2 - 1 Déclaration d'une classe Java	7
2 - 2 Constructeur sans paramètres (constructeur par défaut)	8
2 - 3 Constructeur avec paramètres	8
3 - Modificateurs d'accès	9
3 - 1 - Cas d'une classe	9
3 - 2 - Cas d'un attribut	10
3 - 3 - Cas d'une méthode Java	10
3 - 4 - Le modificateur abstract	10
3 - 5 - Le modificateur Final	10
3 - 6 - - Le modificateur Static	11
C - Héritage et polymorphisme en Java	12
1 - Introduction	12
2 - Héritage Java : étude d'exemple	12
Remarque :	13
Exemple (classe mère)	13
Exemple (classe fille)	13
3 - Polymorphisme en Java	13

Exemple	14
4 - Usage de l'annotation @Override (reformuler une méthode Java)	14
Exemple	15
D - Les interface en Java	16
1 - Qu'est ce qu'une interface en Java ?	16
2 - Une interface est créée pour quelle raison ?	16
3 - Comment créer et utiliser une interface	17
2 - L'instruction try - catch	19
3 - L'exception Java dans sa forme générale en utilisant la classe throwable	20

A- Les tableaux Java - (Java array)



1 - Déclaration des tableaux en Java

Les tableaux sont utilisés pour stocker plusieurs valeurs dans une seule variable, au lieu de déclarer des variables distinctes pour chaque valeur. Pour déclarer un tableau, définissez le type de variable suivie des crochets:

```
String [] laptop;
```

Nous avons maintenant déclaré une variable qui contient un tableau de chaînes. Pour y insérer des valeurs, nous pouvons utiliser un littéral de tableau - placez les valeurs dans une liste séparée par des virgules, entre accolades:

```
String [] laptop = {"Acer", "Hp", "Lenovo"};
```

2 - Accéder aux éléments d'un tableau

Vous accédez à un élément de tableau en vous référant au numéro d'index. Cette déclaration accède à la valeur du premier élément dans le tableau laptop:

Exemple

```
String [] laptop = {"Acer", "Hp", "Lenovo"};  
System.out.println (laptop[0]);  
// Ce qui affiche à l'exécution Acer
```

Remarque:

Les index de tableau commencent par 0: [0] est le premier élément. [1] est le deuxième élément, etc.

3 - Changer un élément du tableau

Pour modifier la valeur d'un élément spécifique, reportez-vous au numéro d'index:

Exemple

```
laptop[0] = "Asus";  
//Exemple  
String [] laptop = {"Acer", "Hp", "Lenovo"};  
laptop[0] = "Asus";  
System.out.println (laptop[0]);  
// Ce qui affiche en sortie maintenant Asus au lieu de Acer
```

4 - Longueur d'un tableau Java

Pour connaître le nombre d'éléments d'un tableau, utilisez la propriété **length**:

Exemple

```
String [] laptop = {"Acer", "Hp", "Lenovo"};  
System.out.println (laptop.length);  
// Résultats 3
```

4 - Boucle à travers un tableau

4-1 Lecture des éléments d'un tableau en utilisant la fonction length()

Vous pouvez parcourir les éléments du tableau avec la boucle for et utiliser la propriété length pour spécifier le nombre de fois que la boucle doit s'exécuter.

L'exemple suivant affiche tous les éléments du tableau laptop:

Exemple

```
String [] laptop = {"Acer", "Hp", "Lenovo"};
for (int i = 0; i < laptop.length; i++) {
    System.out.println (laptop[i]);
}
```

4-2 Lecture des éléments d'un tableau en utilisant la boucle for et une variable string

L'exemple suivant affiche tous les éléments du tableau :

Exemple

```
String [] laptop = {"Acer", "Hp", "Lenovo"};
for (String s: laptop) {
    System.out.println (s);
}
```

L'exemple ci-dessus peut être lu comme ceci: pour chaque élément String (appelé i - comme dans l'index) dans laptop, affichez la valeur de i.

Si vous comparez la boucle for et la boucle for-each, vous verrez que la méthode for-each est plus facile à écrire, qu'elle ne nécessite pas de compteur (à l'aide de la propriété length) et qu'elle est plus lisible.

5 - Tableaux multidimensionnels

Un tableau multidimensionnel est un tableau contenant un ou plusieurs tableaux. Pour créer un tableau à deux dimensions, ajoutez chaque tableau dans son propre ensemble d'accolades:

Exemple

```
int [] [] myTab = {{5, 1,7,21,2}, {5, 6, 7,11}};
```

myTab est maintenant un tableau avec deux tableaux comme éléments.
Pour accéder aux éléments du tableau myTab, spécifiez deux index:

- Le premier pour le tableau
- Le second pour l'élément à l'intérieur de ce tableau.

L'exemple suivant accède au troisième élément (2) du deuxième tableau (1) de myTab:

Exemple

```
int [] [] myTab = {{5, 1,7,21,2}, {5, 6, 9,11}};  
int x = myTab[1][2];  
System.out.println (x); // Sorties 9
```

Nous pouvons également utiliser une boucle for dans une autre boucle for pour obtenir les éléments d'un tableau à deux dimensions (nous devons tout de même pointer vers les deux index):

Exemple

```
public class MyClass {  
    public static void main (String [] args) {  
        int [] [] myTab = {{5, 1,7,21,2}, {5, 6, 9,11}};  
        for(int i = 0; i <myTab.length; ++ i) {  
            for(int j = 0; j <myTab[i].length; ++ j) {  
                System.out.println (myTab[i] [j]); }  
            }  
        }  
    }  
}
```

B - Classes et méthodes Java

1 - Notion d'objets et de classes

1 - 1 Exemple d'une classe prédéfinie : classe String

Le but de la programmation orienté objet est de faciliter la programmation en créant des classes qui à travers auxquelles on crée un nouveau objet chaque fois qu'on a besoin à l'aide de ce qu'on appelle l'instanciation.

Exemple

```
String myString=new String(" salut ");
```

Ici on crée un objet du type chaîne de caractère nommé **myString** à l'aide du modèle **String** la méthode par laquelle on a créé l'objet : **Objet monObjet=new Objet** est appelé **l'instanciation**

Un objet est doté des propriétés et méthodes qui fournissent des informations sur l'objet.

Exemple :

```
myString.length() ;
```

qui fournit ici **la longueur (length en anglais)** de la chaîne de caractères **myString="salut"** qui est égale ici à 5.

Pour créer une classe en Java on utilise la syntaxe :

```
classe nom_classe{  
  //code de la classe ici  
}
```

2 - Les classes et les objets Java

2 - 1 Déclaration d'une classe Java

Pour bien faciliter l'apprentissage nous allons traiter un exemple simple d'une classe permettant de générer des objets voitures ayant tous la même marque et le même prix :

Exemple

```
class voiture{  
    String marque ;  
    double prix ;  
}
```

2 - 2 Constructeur sans paramètres (constructeur par défaut)

Afin de pouvoir créer des objets pour une classe Java, il faut associer à cette dernière une **méthode (fonction)** qui porte le même nom que la classe appelé **constructeur** :

```
voiture(){  
}
```

Nous pouvons aussi attribuer des valeurs aux variables `marque` et `prix` :

```
class voiture {  
    String marque="Renault";  
    double prix=20000;  
    voiture(){  
    }  
}
```

Et maintenant pour créer un objet voiture il suffit de faire l'instanciation : par exemple si on veut créer un objet voiture nommé `maVoiture` on utilise le code :

```
voiture maVoiture = new voiture();
```

L'objet `maVoiture` qu'on vient de créer est une voiture de marque Renault et de prix 20000 (Euro)

Et maintenant si on veut accéder à la marque de l'objet `maVoiture` et de la récupérer sur une variable String on utilise le code :

```
String marque = maVoiture.marque ;
```

Voici le code final de la classe `voiture` :

```
class voiture {  
    String marque="Renault";  
    double prix=20000;  
    voiture(){  
    }  
    public static void main(String[] args) {  
        voiture maVoiture=new voiture();  
        System.out.println("La marque de maVoiture est : "+ maVoiture.marque);  
        System.out.println("Le prix de maVoiture est : "+ maVoiture.prix + "  
Euro");  
    }  
}
```

Ce qui affiche à l'exécution :

La marque de maVoiture est : Renault

Le prix de maVoiture est : 20000.0 Euro

2 - 3 Constructeur avec paramètres

On peut aussi donner la possibilité à l'utilisateur de choisir lui même la marque et le prix de la voiture en ajoutant des paramètres au constructeur :


```

voiture ( String mq , double pr ) {
    this.marque=mq ;
    this.prix=pr ;
}

```

Et maintenant l'utilisateur du modèle `voiture` est libre de choisir la `marque` qu'il souhaite avec le `prix` qui lui convient au moment de l'instanciation , par exemple s'il souhaite créer une voiture de marque **Peugeot** et de **prix 25000 Euro** il n'a qu'à instancier avec ces paramètres :

```

voiture saVoiture=new voiture( "Peugeot" , 25000 ) ;

```

Voici le code final de la classe :

```

class voiture {
    String marque;
    double prix;
    voiture(String mq,double pr){
        this.marque=mq;
        this.prix=pr;
    }
    public static void main(String[] args) {
        voiture saVoiture=new voiture("Peugeot",25000);
        System.out.println("La marque de saVoiture est : "+ saVoiture.marque);
        System.out.println("Le prix de saVoiture est : "+ saVoiture.prix + "
Euro");
    }
}

```

Ce qui affiche à l'exécution :

*La marque de saVoiture est : Peugeot
Le prix de saVoiture est : 25000.0 Euro*

3 - Modificateurs d'accès

Vous auriez sûrement déjà remarqué les mots **public, protected et private** au début des déclarations des méthodes de classe. Ces mots-clés sont appelés les **modificateurs d'accès** dans la syntaxe du langage Java, et ils définissent la portée d'un constituant de la classe.

3 - 1 - Cas d'une classe

- Si une classe a une visibilité **public** (publique), la classe peut être appelée depuis n'importe où dans le programme.

- Si une classe possède une visibilité du package, la classe ne peut être appelée que dans le package où la classe est définie.
- Si une classe a une visibilité **private** (privée) (elle ne peut se produire que si la classe est définie imbriquée dans une autre classe), la classe ne peut être consultée que dans la classe externe.

3 - 2 - Cas d'un attribut

- Si une variable est définie dans une **classe publique** et qu'elle a une visibilité **public**, la variable peut être appelée n'importe où dans l'application par la classe dans laquelle elle est définie.
- Si une variable a une visibilité **protected** (protégée), la variable ne peut être appelée que dans les **sous-classes** et dans le même paquet dans la classe dans laquelle elle est définie.
- Si une variable a une visibilité du package, la variable ne peut être référencée que dans le même paquet dans la classe dans laquelle elle est définie.
- Si une variable a une visibilité **private** (privée), la variable ne peut être accessible que dans la classe dans laquelle elle est définie.

3 - 3 - Cas d'une méthode Java

- Si une méthode est définie dans une **classe publique** et qu'elle a une **visibilité publique**, la méthode peut être appelée n'importe où dans l'application à travers la classe dans laquelle elle est définie.
- Si une méthode est déclarée **protected** elle a une visibilité protégée, la méthode peut être appelée uniquement dans les sous-classes et dans le même paquetage à travers la classe dans laquelle elle est définie.
- Si une méthode est déclarée **private** elle a une visibilité privée, la méthode peut être appelée uniquement dans la classe dans laquelle elle est définie.

3 - 4 - Le modificateur abstract

Le modificateur **abstract** est utilisé devant les classes Java pour définir les **classes abstraites** (les classes abstraites sont des classes non instanciables et par suite non exécutables directement, elles ne peuvent être utilisées qu'en héritage)

3 - 5 - Le modificateur Final

- En Java le mot clé **final** s'il est ajouté devant un **attribut**, il le rend non modifiable, il prend toujours la valeur par laquelle il est initialisé.
- Le mot clé **final** s'il est ajouté devant une **méthode**, il indique que cette dernière ne peut pas être modifiée pendant l'héritage. Les méthodes **static** et **private** sont considérées automatiquement **final**.
- Le mot clé **final** s'il est ajouté devant une **classe**, il indique que cette dernière ne peut pas avoir de **classes filles**.

3 - 6 - - Le modificateur Static

Pour un attribut ou une méthode Java, le modificateur **static** indique, que la méthode ou l'attribut peut être appelé sans faire une instantiation sur sa classe.

Dr KANGA Koffi

C - Héritage et polymorphisme en Java

1 - Introduction

L'héritage en Java est un processus dans lequel une classe acquiert les propriétés (attributs et méthodes) d'une autre classe Java. Avec l'utilisation de l'héritage, les données et informations sont gérées dans un ordre hiérarchique bien organisé et structuré.

La classe qui hérite des propriétés d'une autre classe est appelée sous-classe (classe dérivée ou classe fille) et la classe dont les propriétés sont héritées est appelée superclasse (classe de base, classe parente, classe mère...).

La déclaration d'une sous classe se fait à l'aide du mot clé **Extends**

Extends est le mot clé utilisé pour hériter des propriétés d'une classe.

Syntaxe

```
class mère {  
    ....  
}  
class fille extends mère {  
    ....  
}
```

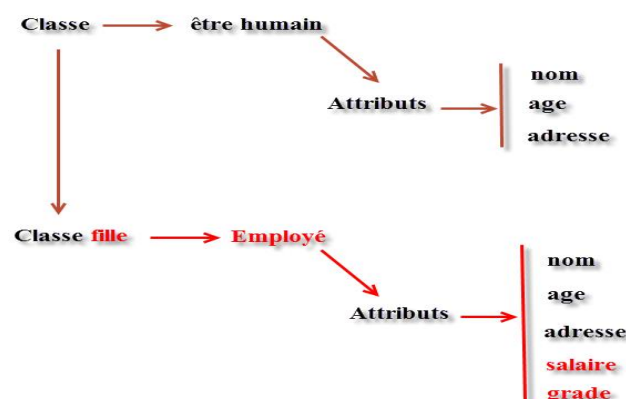
2 - Héritage Java : étude d'exemple

Pour bien comprendre l'héritage Java, nous allons traiter un exemple simple d'une classe **être humain** et d'une sous classe **Employé**

La classe être humain est dotée des attributs :

- nom
- age
- adresse

Un **employé** est aussi un **être humain** et par suite la sous classe **employé** hérite les mêmes propriétés **nom, age, adresse...** de la classe mère **être humain**, néanmoins la classe fille **employé** possède d'autres attributs inexistants chez la classe mère comme **salaire, grade...**



Remarque :

L'héritage d'un attribut se fait grâce au mot clé **super**

Exemple (classe mère)

```
package etreHumain;

public class Humain {
    public String nom;
    public Humain(String n){
        this.nom=n;
    }
}
```

Exemple (classe fille)

```
package etreHumain;

class Employé extends Humain{
    public int salaire;

    public Employé(String nm, int sal) {
        super(nm);
        this.salaire=sal;
    }

    public static void main(String[] args) {
        Employé enseignant=new Employé("Robert",3500);
        System.out.println("Le nom de l'enseignant est " + enseignant.nom);
        System.out.println("Salaire de l'enseignant: " + enseignant.salaire + " Fcfa");
    }
}
```

Ce qui affiche après exécution :

Le nom de l'enseignant est Robert

Le salaire de l'enseignant est 3500 Fcfa

3 - Polymorphisme en Java

Dans le paragraphe précédent , nous avons parlé de superclasses et de sous-classes. Si une classe hérite d'une méthode de sa superclasse, alors il y a une chance de reformuler la méthode à condition qu'il n'est pas marqué final.

L'avantage de la substitution est la capacité de définir un comportement spécifique au type de sous-classe, ce qui signifie qu'une sous-classe peut implémenter une méthode de classe parent en fonction de son besoin et ses exigences.

Exemple

```
public class Animal {
    public void seDeplacer() {
        System.out.println("Un animal peut se déplacer");
    }
}

class Chien extends Animal {
    public void seDeplacer() {
        System.out.println("Un chien peut courir");
    }
}

class Oiseau extends Animal {
    public void seDeplacer() {
        System.out.println("Un oiseau peut voler");
    }
}
```

Et pour exécuter le code de cette classe nous aurons besoin d'une autre classe dotée d'une méthode **static void main()**

```
public class ExecutionAnimal {

    public static void main(String args[]) {
        Animal a = new Animal();
        Animal b = new Chien();
        Animal c = new Oiseau();

        a.seDeplacer();
        b.seDeplacer();
        c.seDeplacer();
    }
}
```

Ce qui affiche après exécution sur Eclipse :

Un animal peut se déplacer
Un chien peut courir
Un oiseau peut voler

4 - Usage de l'annotation @Override (reformuler une méthode Java)

L'annotation @Override est utilisée en Java pour reformuler une méthode dans le cas du polymorphisme Java. Si la formule en question ne redéfinit pas une autre dans la classe mère, le compilateur signale une erreur !

Exemple

```
class Parent {  
  
    void afficher(){  
        System.out.println("C'est la méthode de la classe Parent ");  
    }  
}  
  
class Enfant extends Parent{  
  
    @Override  
    void afficher(){  
        System.out.println("C'est la méthode de la classe Enfant ");  
    }  
}
```

@Override indique ici au compilateur que la classe **Enfant** utilise la méthode **afficher()** de la classe **Parent** mais avec modification : au lieu d'afficher le message : **C'est la méthode de la classe Parent** elle doit afficher un autre qui est propre à la classe **Enfant** : **C'est la méthode de la classe Enfant**.

Certainement vous n'avez pas pu comprendre réellement l'avantage et l'effet de cette annotation !

Comme nous l'avons déjà mentionné ci-dessus : **Si la formule en question ne redéfinit pas une autre dans la classe mère, le compilateur signale une erreur !. Amusons nous par exemple faire une petite modification sur le nom de la formule en question :**

Essayons à titre d'exemple de remplacer la méthode **afficher()** dans la classe Enfant par **affiche()**;

```
3  class Parent {  
4  
5  void afficher(){  
6      System.out.println("C'est la méthode de la classe Parent ");  
7  }  
8  }  
9  
10 class Enfant extends Parent{  
11  
12  @Override  
13  void affiche(){  
14      System.out.println("C'est la méthode de la classe Enfant ");  
15  }  
16  }
```

D - Les interface en Java



1 - Qu'est ce qu'une interface en Java ?

Une interface du langage de programmation Java est un type abstrait utilisé pour spécifier un comportement que les classes doivent implémenter. Ils sont similaires aux protocoles. Les interfaces sont déclarées à l'aide du mot-clé `interface` et ne peuvent contenir que des déclarations de méthode et des déclarations de constante (variable déclarative déclarée à la fois statique et finale). Toutes les méthodes d'une interface ne contiennent pas d'implémentation (corps de méthodes) de toutes les versions antérieures à Java 11. À partir de Java 8, les méthodes par défaut et les méthodes statiques peuvent avoir une implémentation dans la définition d'interface.

Les interfaces ne peuvent pas être instanciées, mais sont plutôt implémentées. Une classe qui implémente une interface doit implémenter toutes les méthodes non définies par défaut décrites dans l'interface ou être une classe abstraite. Les références d'objet en Java peuvent être spécifiées comme étant d'un type d'interface; dans chaque cas, ils doivent être nuls ou être liés à un objet qui implémente l'interface.

L'un des avantages de l'utilisation des interfaces est qu'elles simulent un héritage multiple. Toutes les classes en Java doivent avoir exactement une classe de base, la seule exception étant `java.lang.Object` (la classe racine du système de types Java); l'héritage multiple de classes n'est pas autorisé. Cependant, une interface peut hériter de plusieurs interfaces et une classe peut implémenter plusieurs interfaces.

2 - Une interface est créée pour quelle raison ?

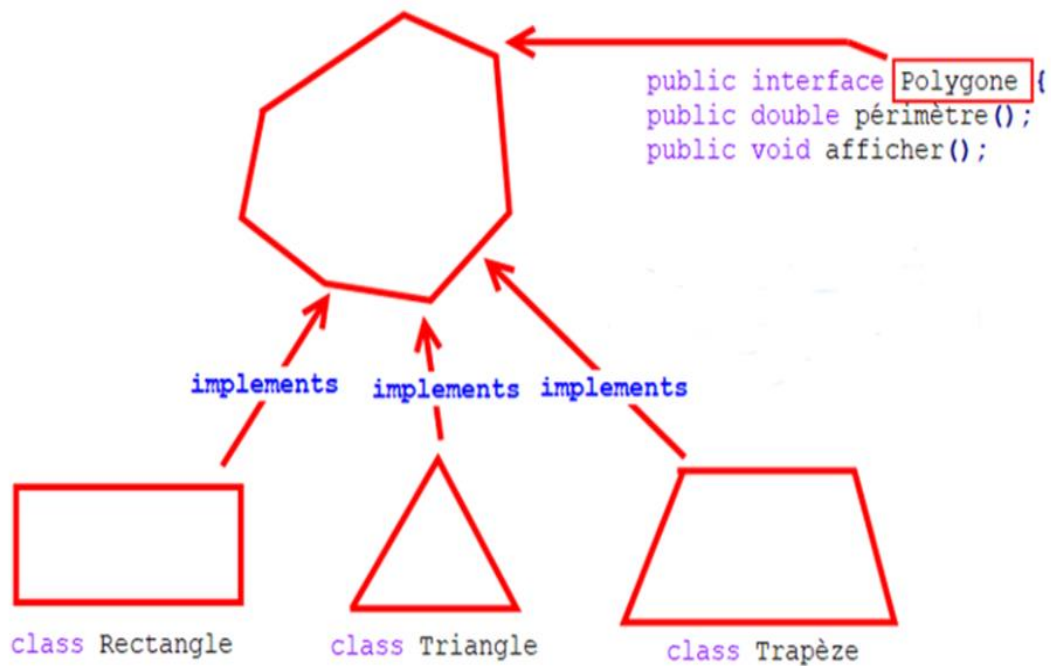
Pour bien comprendre l'utilité des interfaces en Java, prenons un exemple simple :

- On souhaite par exemple créer une **classe Rectangle** qui permet de générer un objet **rectangle**
- On souhaite aussi créer une **classe triangle** permettant de générer un objet **triangle**
- On souhaite créer une **classe trapèze** permettant de générer un objet **trapèze**
-

Les objets **rectangle**, **triangle** et **trapèze** possèdent tous **des propriétés communes** comme : **surface**, **perimètre**, **nombre de cotés**... Et on souhaite créer un moyen qui permet de gérer ce processus c.a.d **un triangle** doit **obligatoirement** posséder

une méthode qui **calcule le périmètre** et une **méthode qui calcul la surface**, faute de quoi le développeur rencontre un message d'erreur lui indiquant qu'il doit obligatoirement insérer les méthodes **perimetre()** et **surface()**

C'est à ce niveau là qu'une interface intervient ! On doit créer une et appelons la par exemple interface polygone qui contient l'ensemble des méthodes communes aux formes géométriques cités ci-dessus :



3 - Comment créer et utiliser une interface

En pratique, la création et la déclaration d'une interface est semblable à une **classe**, à la différence que l'on remplace le mot-clé **class** par **interface**.

```
public interface Polygone {
    public double périmètre();
    public int nombreCotes();
}
```

Les classes **rectangle**, **triangle** et **trapèze** vont être maintenant déclarées de la même façon comme celle de l'héritage, à la différence de remplacer le mot clé **extends** par **implements**:

Class Rectangle

```
public class Rectangle implements Polygone {
    public double Longueur;
    public double largeur;
    public Rectangle(double L, double l){
        this.Longueur=L;
        this.largeur=l;
    }

    public double périmètre(){
        return 2*(this.Longueur+this.largeur);
    }
}
```

```

    }
    public int nombreCotes(){
        return 4;
    }

    public void afficher(){
System.out.println("La périmètre du rectangle est : "+ this.périmètre());
    }
    public static void main(String[] args) {
        Polygone R1=new Rectangle(7,4);
        Rectangle R2=new Rectangle(9,3);
        R1.afficher();
        R2.afficher();
    }
}

Class Triangle
public class Triangle implements Polygone{
    public double coté1;
    public double coté2;
    public double coté3;
    //création du constructeur
    public Triangle(double c1,double c2,double c3){
        this.coté1=c1;
        this.coté2=c2;
        this.coté3=c3;
    }

    public double périmètre(){
        return this.coté1+this.coté2+this.coté3;
    }
    public int nombreCotes(){
        return 3;
    }
    public void afficher(){
System.out.println("La périmètre du triangle est : "+ this.périmètre());
    }
    public static void main(String[] args) {
        Polygone T1=new Triangle(2,3,5);
        Triangle T2=new Triangle(4,4,5);
        T1.afficher();
        T2.afficher();
    }
}

```

1 - Introduction aux exceptions Java

Lors de l'exécution du code Java, différentes erreurs peuvent se produire:

- erreurs de codage commises par le programmeur,
- erreurs de l'utilisateur dues à une entrée erronée,
- fichier introvable... ou autre chose imprévisible....

En cas d'erreur, Java s'arrête normalement et génère un message d'erreur. Le terme technique utilisé est le suivant: Java générera une exception (une erreur).

Heureusement le langage Java est doté d'une instruction **try - catch** :

2 - L'instruction try - catch

- L'instruction **try** vous permet de définir un bloc de code à tester et qu'il ne sera traité par Java que s'il ne contient pas d'erreur ! En cas d'erreur, Java se dirige automatiquement au bloc **catch**:
- L'instruction **catch** vous permet de définir un bloc de code à exécuter si une erreur se produit dans le bloc **try**, et qui pourra sauver l'affaire en remplaçant le morceau de code erroné qui se trouve dans le bloc **try**

Les mots clés **try** and **catch** viennent par paires:

Syntaxe

```
try {  
    // Bloc de code à tester  
} catch (Exception e) {  
    // Bloc de code à traiter en cas d'erreur dans le bloc try  
}
```

Prenons l'exemple suivant:

Cela générera une erreur, car **myClass[5]** n'existe pas.

```
public class myClass {  
    public static void main(String[] args) {  
        String [] pc = {"Acer", "Azus", "Hp"};  
        System.out.println (pc [5]); // Erreur!  
        //En plus l'exécution du programme s'arrête !  
        System.out.println ("voici un message qui ne sera pas exécuté à cause de  
l'erreur !");  
    }  
}
```

La sortie ressemblera à ceci :

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
at exceptionTest.Except.main(Except.java:8)

Comme vous le voyez un message méchant et horrible ! En plus l'exécution du programme s'arrête ! Vous avez bien remarqué que le dernier output n'a pas été exécuté.

Pour régler ce problème, nous allons utiliser l'instruction **try - catch**

```
public static void main(String[] args) {  
  
    try{ /*ici on demande à Java de faire seulement un essai de traiter ce code et de  
l'ignorer en cas d'erreur pour passer au 2ème code qui se trouve dans le bloc catch  
*/  
        String [] pc = {"Acer", "Azus","Hp"};  
        System.out.println (pc [5]); // Erreur!  
    }  
    catch(Exception e){/*Java traitera ce code seulement si le code qui  
se trouve dans le bloc try contient une erreur ! */  
        System.out.println("Votre choix n'est pas valide !");  
    }  
    System.out.println ("L'exécution du programme se poursuit maintenant !");  
}
```

Ce qui affiche à l'exécution un gentil petit message :

Votre choix n'est pas valide !

L'exécution du programme se poursuit maintenant !

Et en plus l'exécution du programme se poursuit sans problème !

3 - L'exception Java dans sa forme générale en utilisant la classe throwable